Logan Titzer
CS379

PA 7 The Parameter Simulator

1. Three Modes, One Function

For this one, the function swaps a and b using a temporary variable t. The caller starts with x = 1 and y = 2, but the final result depends on how the parameters are passed.

```
swap(a, b) {
let t = a;
a = b;
b = t;
}
```

Pass-by-value

With pass-by-value, a and b get their own copies, so changing them inside the function does not change x and y in the caller.

Before the call: x = 1, y = 2. There is no callee frame yet.
After binding: a = 1, b = 2.
After t = a: a = 1, b = 2, t = 1. The caller is still x = 1, y = 2.
After a = b: a = 2, b = 2, t = 1. The caller is still x = 1, y = 2.
After b = t: a = 2, b = 1, t = 1. The caller is still x = 1, y = 2.
After return: The callee frame is removed. The caller still has x = 1, y = 2.

So the caller still has x = 1 and y = 2. Only the local copies got swapped.

Pass-by-reference

With pass-by-reference, a and b point straight to the caller's variables. That means assignments in the function show up in x and y right away.

Before the call: x = 1, y = 2. There is no callee frame yet.
After binding: a points to x, and b points to y.
After t = a: t = 1. The caller is still x = 1, y = 2.
After a = b: x becomes 2, so x = 2, y = 2.
After b = t: y becomes 1, so x = 2, y = 1.
After return: The callee frame is removed. The caller has x = 2, y = 1.

The caller ends with x = 2 and y = 1 because a and b use the caller's actual variable cells.

Pass-by-value-result

With value-result, a and b start as copies just like pass-by-value. The difference is that their final values get copied back when the function returns, first a and then b.

Before the call: x = 1, y = 2. There is no callee frame yet.
After binding: a = 1, b = 2.
After t = a: a = 1, b = 2, t = 1. The caller is still x = 1, y = 2.
After a = b: a = 2, b = 2, t = 1. The caller is still x = 1, y = 2.
After b = t: a = 2, b = 1, t = 1. The caller is still x = 1, y = 2.
Copy back a: x becomes 2, so x = 2, y = 2.
Copy back b: y becomes 1, so x = 2, y = 1.
After return: The callee frame is removed. The caller has x = 2, y = 1.

The caller does not see the changes while the function is running; it only gets the final values when they are copied back.

2. Asymmetric Aliasing

Here, P(x, y) increments both parameters, and the call is P(a, a) with a starting at 10. The important part is that both arguments refer to the same caller variable.

```
P(x, y) {
x = x + 1;
y = y + 1;
}
```

Pass-by-reference

Both x and y point to the same caller variable a, so the two increments build on each other.

Before the call: a = 10.

After binding: x and y both point to a.

After x = x + 1: a = 11. Both names now see 11.

After y = y + 1: a = 12. The second increment starts from 11.

After return: The final caller value is a = 12.

Pass-by-value-result

With value-result, x and y have separate local cells, both starting at 10. They are copied back in parameter order, x first and then y.

Before the call: a = 10.

After binding: x = 10 and y = 10 in separate local cells.

After x = x + 1: x = 11, y = 10. The caller's a is still 10.

After y = y + 1: x = 11, y = 11. The caller's a is still 10.

Copy back x: The caller's a becomes 11.

Copy back y: The caller's a is set to 11 again.

After return: The final caller value is a = 11.

This is why the modes give different answers for P(a, a). Reference mode uses one shared cell, so the increments reach 12. Value-result uses two separate local cells, and both values copied back are 11.

3. Closures and Capture

The code makes three functions inside a loop:

```
let makers = [];
for (i = 0; i < 3; i = i + 1) {
makers.push(func() { return i; });
}
```

All three closures return 3 when they are called after the loop finishes, assuming closures capture the environment rather than saving a separate copy of i when each closure is created.

The reason is that all three closures use the same i binding. The loop keeps updating that binding, and when it ends, i is 3. When a closure runs, its activation record follows the static link back to the environment where the closure was created. Since the closures share that environment, they all find the same final value of i.

The smallest change would be to make the for loop create a new i binding and cell for every iteration. Each closure would then capture its own iteration's value, so the three closures would return 0, 1, and 2. This changes the for loop's behavior, not how closures work.